UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

> Reinventa el mundo <

Pilas y colas

Semana 04

Brenner Ojeda
bojeda@utec.edu.pe

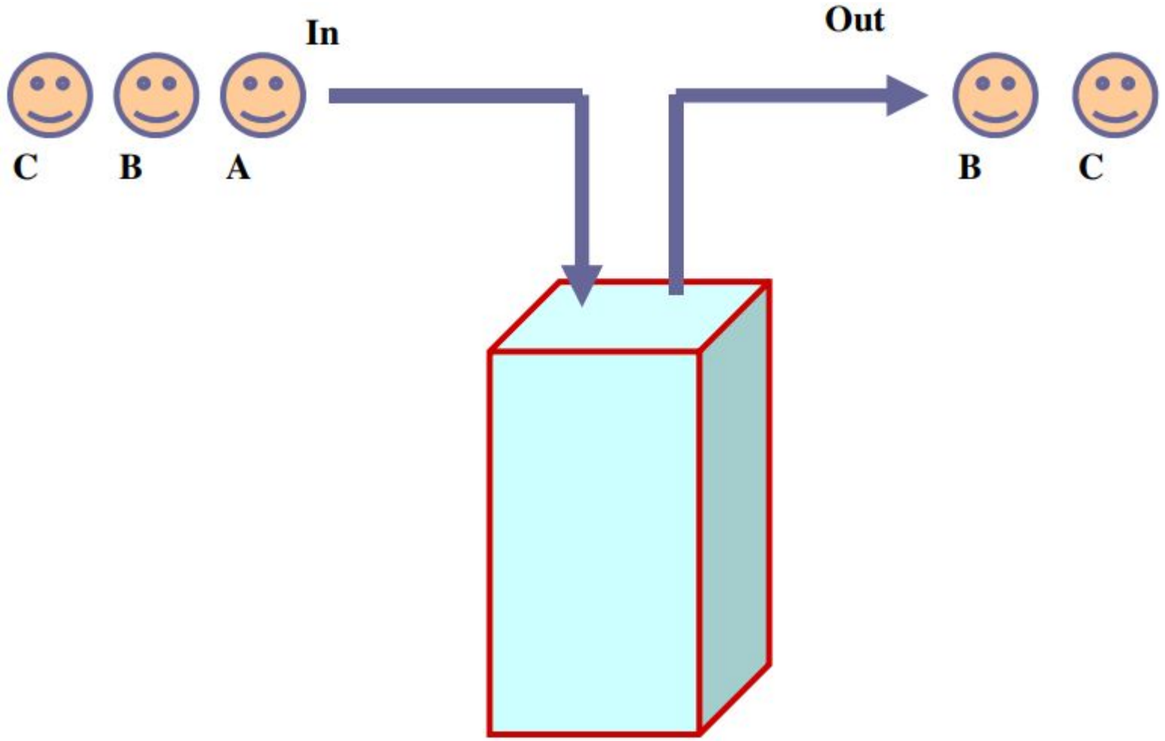
TRANSFORMATEC

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Pilas

TRANSFORMATEC

Estructura de datos con comportamiento *último en entrar, primero en salir* (LIFO)



Operaciones típicas sobre stacks

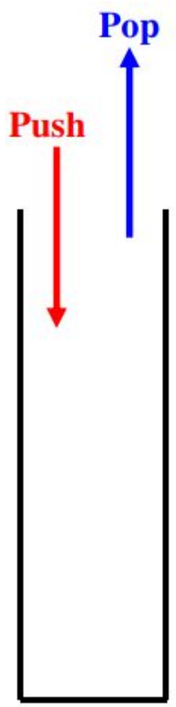
isempty: determines if the stack has no elements

isfull: determines if the stack is full in case of a bounded sized stack

top: returns the top element in the stack

push: inserts an element into the stack

pop: removes the top element from the stack



push is like inserting at the front of the list

pop is like deleting from the front of the list

Crear e inicializar una pila

Declaration

```
#define MAX_STACK_SIZE 100
typedef struct {
    int key; /* just an example, can have
                any type of fields depending
                on what is to be stored */
}  element;
typedef struct {
    element list[MAX_STACK_SIZE];
    int top; /* index of the topmost element */
 } stack;
```

Create and Initialize

```
stack Z;
Z.top = -1;
```

Operaciones

```
int isfull (stack *s)
{
    if (s->top >=
            MAX_STACK_SIZE – 1)
        return 1;
    return 0;
}
```

```
int isempty (stack *s)
{
    if (s->top == -1)
        return 1;
    return 0;
}
```

Operaciones

```
element top( stack *s )
{
    return s->list[s->top];
}
```

```
void pop( stack *s )
{
    (s->top)--;
}
```

```
void push( stack *s, element e )
{
    (s->top)++;
    s->list[s->top] = e;
}
```

Aplicación: Coincidencia de paréntesis

Dada una expresión paréntesis, pruebe si la la expresión está correctamente entre paréntesis. Ejemplo:

( )({ } [({ } { } ( ))])	is proper
( ){ [ ]	is not proper
({) }	is not proper
)([ ]	is not proper
([ ]))	is not proper

Approach

- Siempre que se encuentra un paréntesis izquierdo, se inserta en la pila.
- Siempre que encuentre un paréntesis derecho, sáquelo de la pila y verifique si los paréntesis coinciden.
- Funciona para múltiples tipos de paréntesis (), {}, []

Parenthesis matching

```
while (not end of string) do
{
    a = get_next_token();
    if (a is '(' or '{' or '[') push (a);
    if (a is ')' or '}' or ']')
    {
        if (is_stack_empty( ))
          { print ("Not well formed"); exit(); }
        x = top();
        pop();
        if (a and x do not match)
          { print ("Not well formed"); exit(); }
    }
}
if (not is_stack_empty( )) print ("Not well formed");
```

Recursion can be implemented as a stack

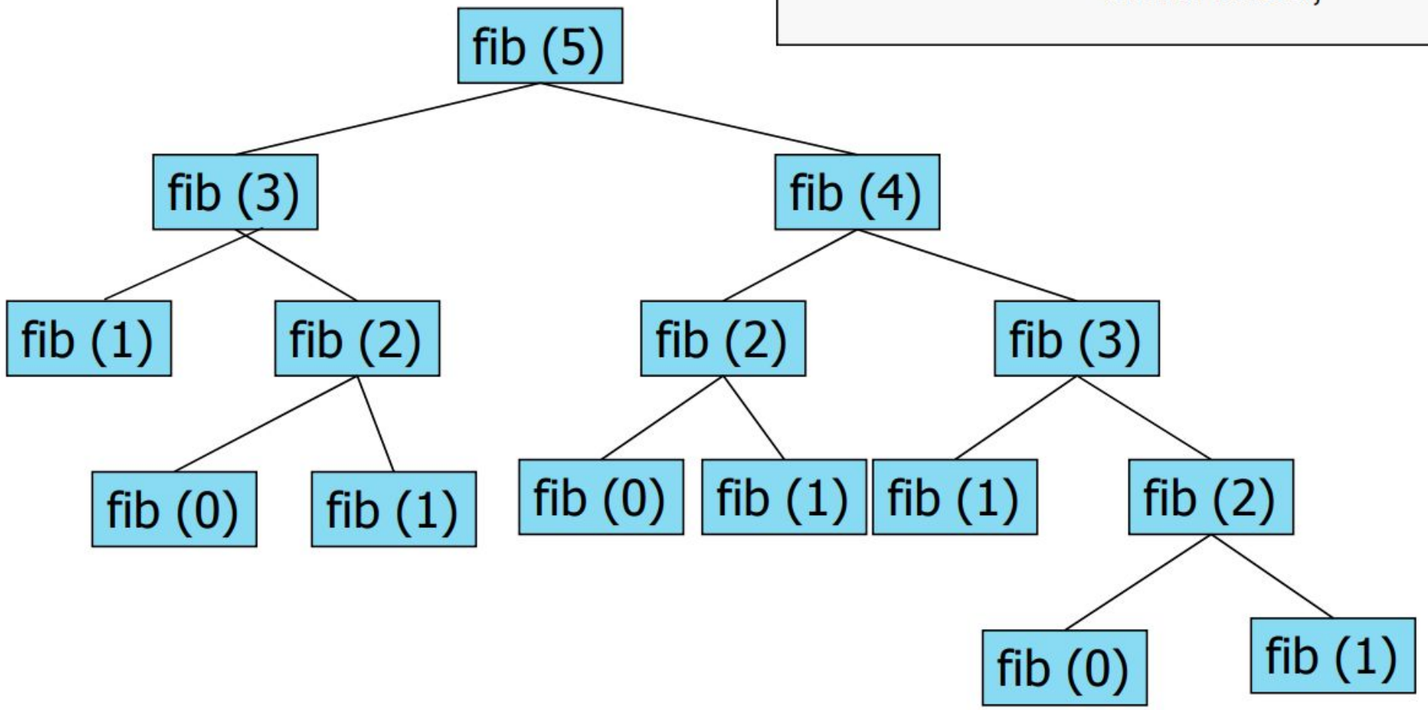
```
int factorial_recursive(int n) {
    if (n == 0 || n == 1)
        return 1;
    else
        return n * factorial_recursive(n - 1);
}
```

```
int factorial_with_stack(int n) {
    if (n == 0 || n == 1)
        return 1;

    stack<int> factorials;
    factorials.push(1);

    for (int i = 2; i <= n; ++i) {
        factorials.push(i * factorials.top());
    }

    return factorials.top();
}
```

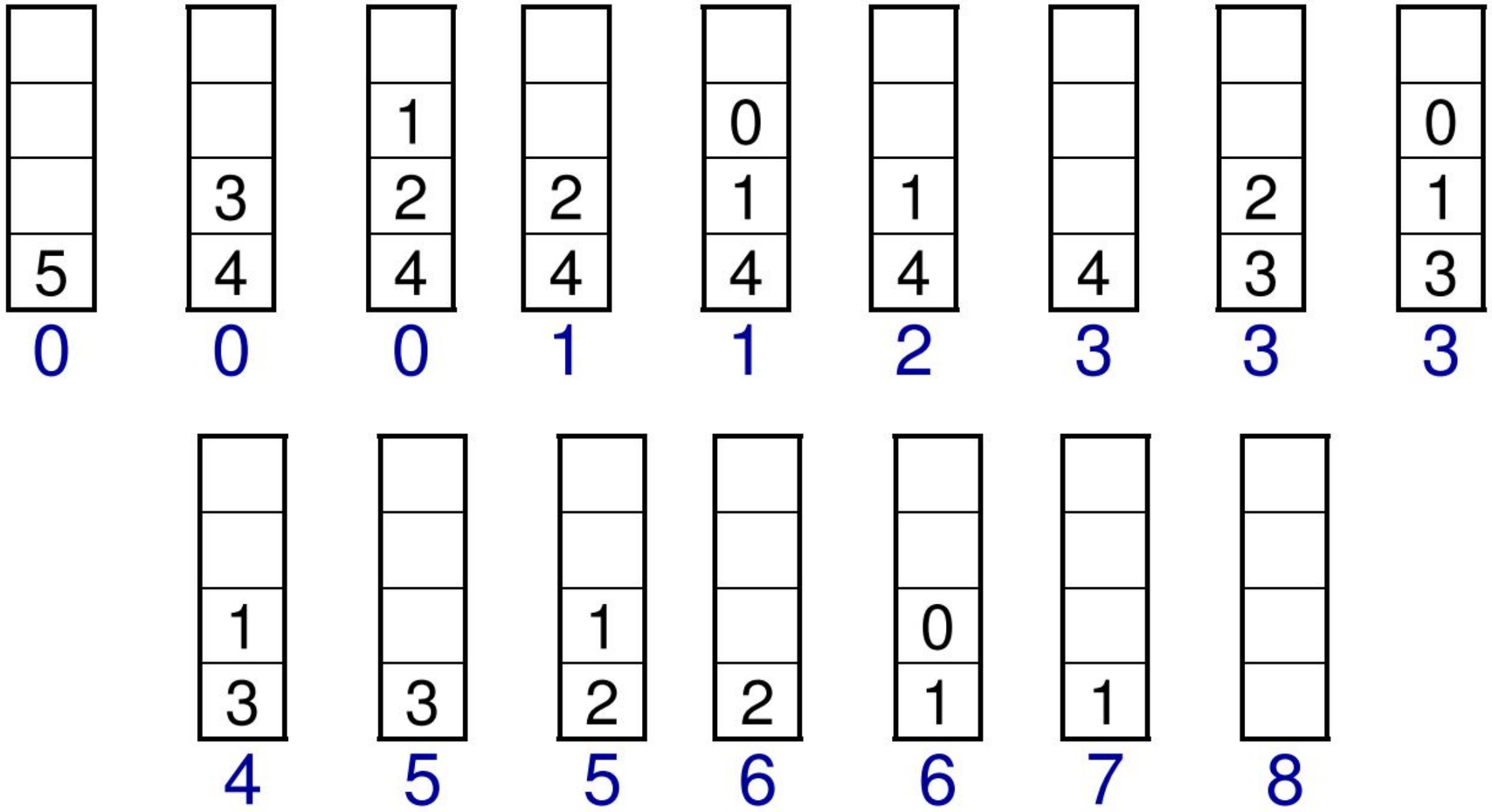
Recursion can be implemented as a stack

Fibonacci recurrence:

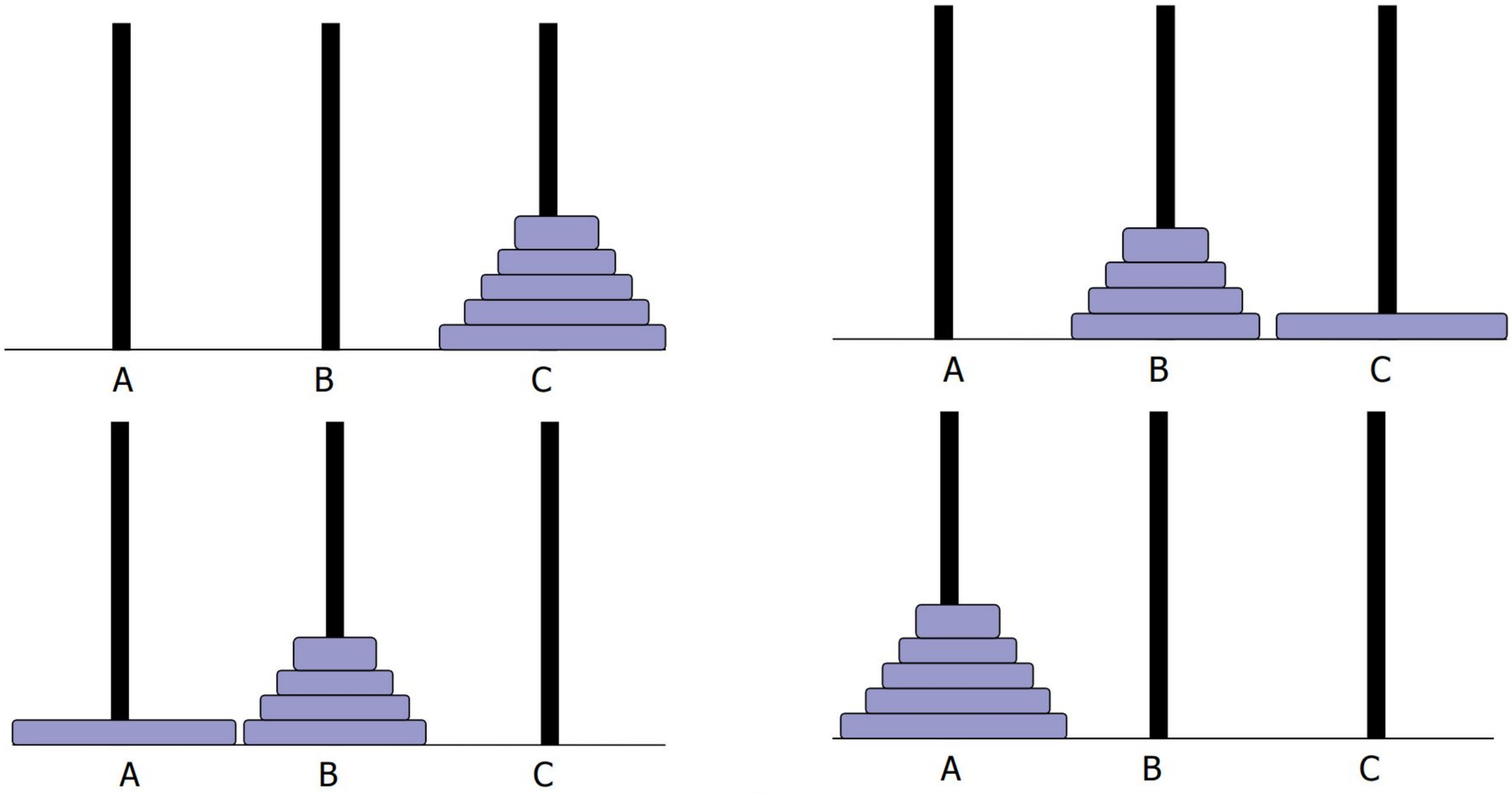
fib(n) = 1 if n =0 or 1;
= fib(n – 2) + fib(n – 1)
otherwise;



Pila de recursividad de Fibonacci



Torre de Hanoi



Torre de Hanoi

```
void hanoi(int n, char from_rod, char to_rod, char aux_rod) {
    if (n == 1) {
        cout << "Move disk 1 from " << from_rod << " to " << to_rod << std::endl;
        return;
    }
    hanoi(n - 1, from_rod, aux_rod, to_rod);
    cout << "Move disk " << n << " from " << from_rod << " to " << to_rod << std::endl;
    hanoi(n - 1, aux_rod, to_rod, from_rod);
}
```

```
int main() {
    int num_disks = 3; // Cambia este valor para el número deseado de discos
    hanoi(num_disks, 'A', 'C', 'B'); // 'A', 'B' y 'C' son los nombres de las varillas
    return 0;
}
```

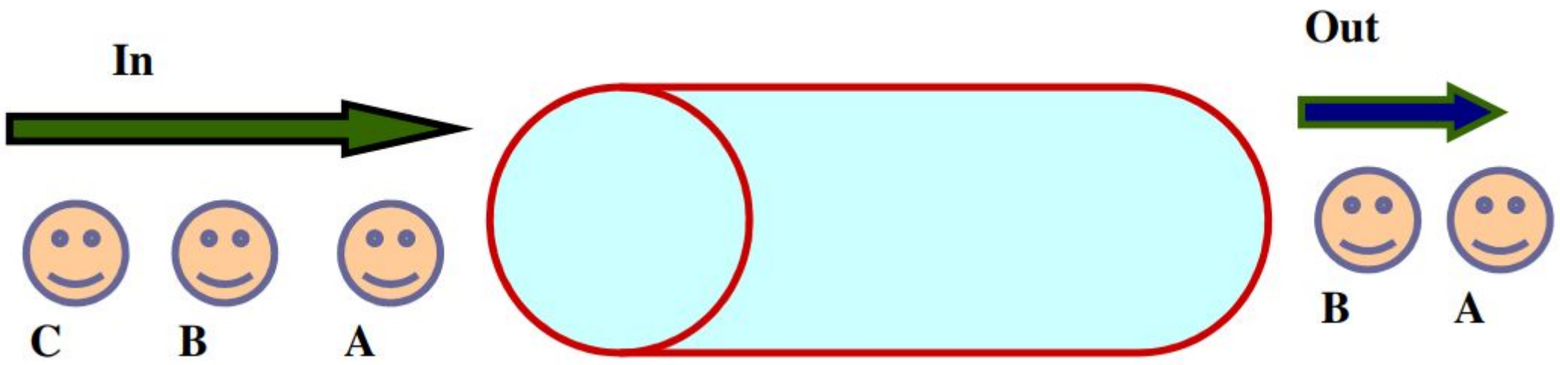
Puedes implementar el algoritmo de las Torres de Hanoi utilizando pilas?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Colas

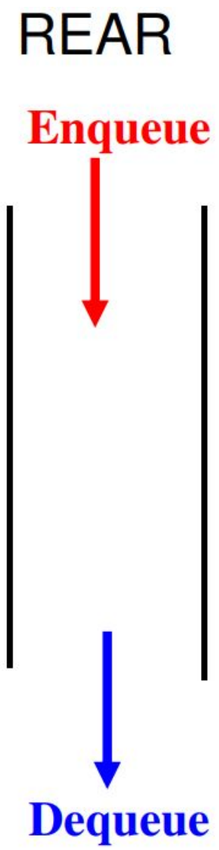
TRANSFORMATEC

Estructura de datos con comportamiento primero en entrar, primero en salir (FIFO)



Operaciones típicas sobre Colas

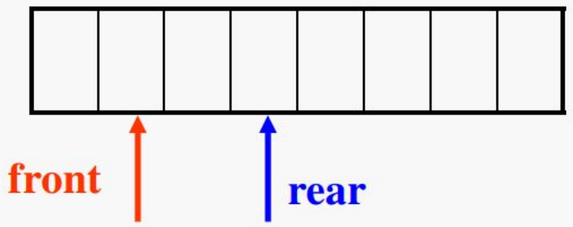
isempty: determines if the queue is empty
isfull: determines if the queue is full
in case of a bounded size queue
front: returns the element at front of the queue
enqueue: inserts an element at the rear
dequeue: removes the element in front



Posibles implementaciones

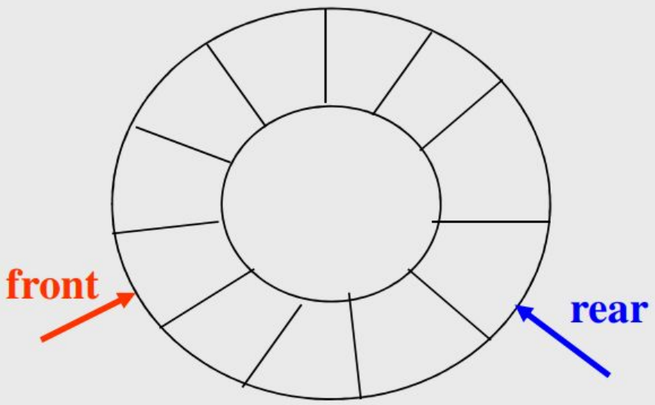
Linear Arrays:

(static/dynamicaly allocated)



Circular Arrays:

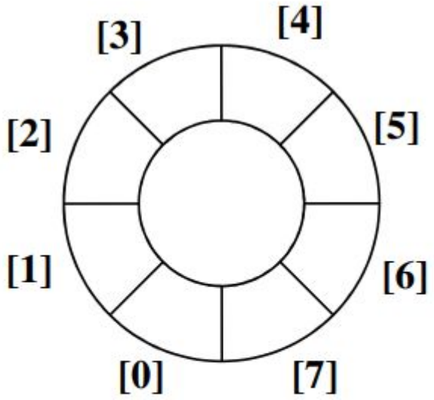
(static/dynamically allocated)



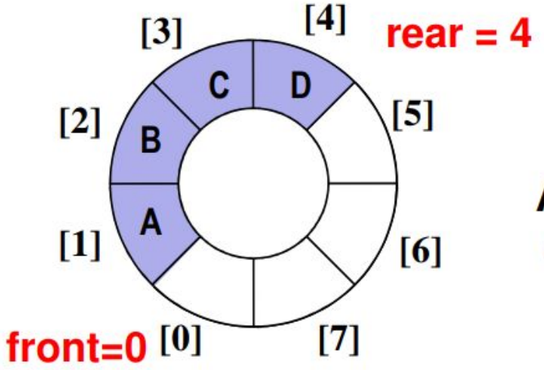
Can be implemented by a 1-d array using modulus operations

Linked Lists: **Use a linear linked list with insert_rear and delete_front operations**

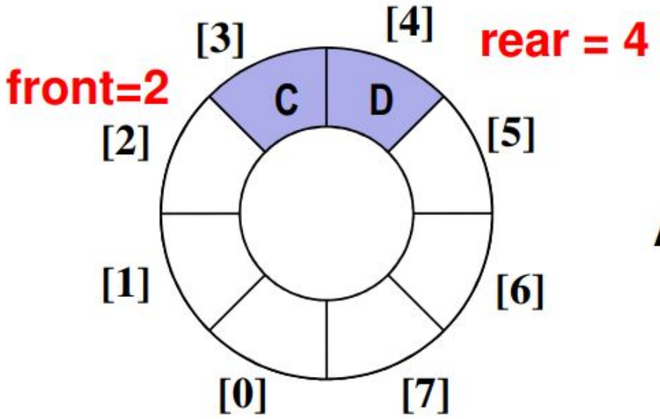
Circular Queue



front=0
rear=0

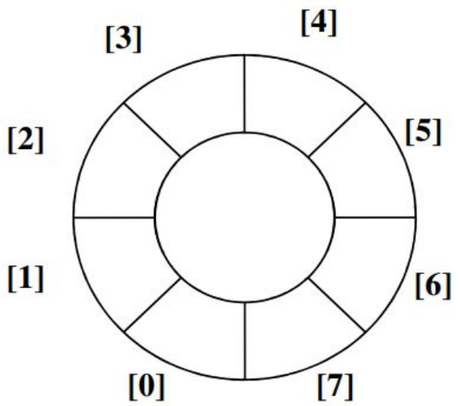


After insertion
of A, B, C, D



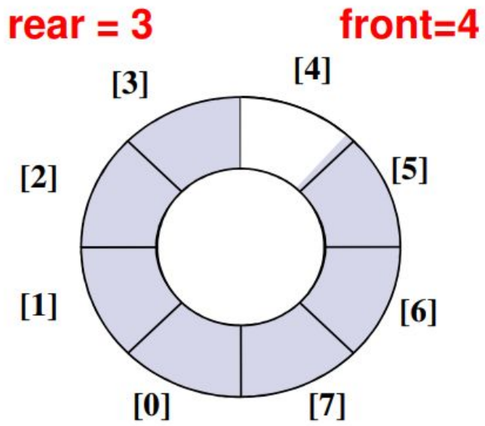
After deletion of
of A, B

front: index of queue-head (always empty – why?)
rear: index of last element, unless rear = front



front=0
rear=0

Queue Empty



Queue Full

Queue Empty Condition: ***front == rear***
Queue Full Condition: ***front == (rear + 1) % MAX_Q_SIZE***

Crear e inicializar una cola circular

Declaration

```
#define MAX_Q_SIZE 100
typedef struct {
    int key; /* just an example, can have
                any type of fields depending
                on what is to be stored */
}  element;
typedef struct {
    element list[MAX_Q_SIZE];
    int front, rear;
 } queue;
```

Create and Initialize

```
queue Q;
Q.front = 0;
Q.rear = 0;
```

Operaciones

```
int isfull (queue *q)
{
    if (q->front == ((q->rear + 1) %
                        MAX_Q_SIZE))
        return 1;
    return 0;
}
```

```
int isempty (queue *q)
{
    if (q->front == q->rear)
        return 1;
    return 0;
}
```

Operaciones

```
element front( queue *q )
{
    return q->list[(q->front + 1) % MAX_Q_SIZE];
}
```

```
void enqueue( queue *q, element e)
{
   q->rear = (q->rear  + 1)%
                    MAX_Q_SIZE;
   q->list[q->rear] = e;
}
```

```
void dequeue( queue *q )
{
   q-> front =
         (q-> front + 1)%
                  MAX_Q_SIZE;
}
```

Recuerden...

El curso se enfoca en que entiendan cada estructura de datos, sus algoritmos y usos comunes.

No hay que reinventar la rueda, en el mercado lo más probable es que utilicen otras implementaciones, por ejemplo de la Standard Template Library (STL).

De todas formas, habrá situaciones en las que probablemente implementarán sus propias estructuras.